

Presentación ejecutiva

Diseño conceptual y lógico de base de datos para Ecommify

Arquitectura Transaccional-Analítica | PostgreSQL + MongoDB

Maestría en Arquitectura de Software · Diseño y Optimización de Bases de Datos

1. Contexto y objetivo

E-commerce multivendedor con cargas transaccionales y analíticas.

Contexto

Ecommify gestiona clientes, vendedores, productos, pedidos, pagos, reseñas, inventario y geolocalización.

Dataset base

Brazilian E-Commerce / Olist: 9 tablas CSV para análisis de estructura, relaciones, calidad y volumen.

Objetivo

Diseñar un modelo conceptual y lógico implementable en PostgreSQL, con módulo NoSQL preliminar en MongoDB.

- Procesamiento OLTP: pedidos, pagos e inventario con consistencia fuerte.
- Procesamiento OLAP: tableros de ventas, clientes, categorías y comportamiento.
- Enfoque académico: prototipo validable en Supabase, MongoDB Atlas y Colab.

OLTP

OLAP

**Arquitectura híbrida
para diferenciar patrones
de acceso y criticidad**

2. Requisitos clave

Síntesis de requisitos funcionales y no funcionales medibles.

Clientes y vendedores

Registro, ubicación y datos comerciales.

Productos

Categorías, dimensiones, fotos y especificaciones variables.

Pedidos y pagos

Órdenes, ítems, estados, cuotas y valores.

Analítica

Ventas por categoría, mes, cliente, vendedor y estado.

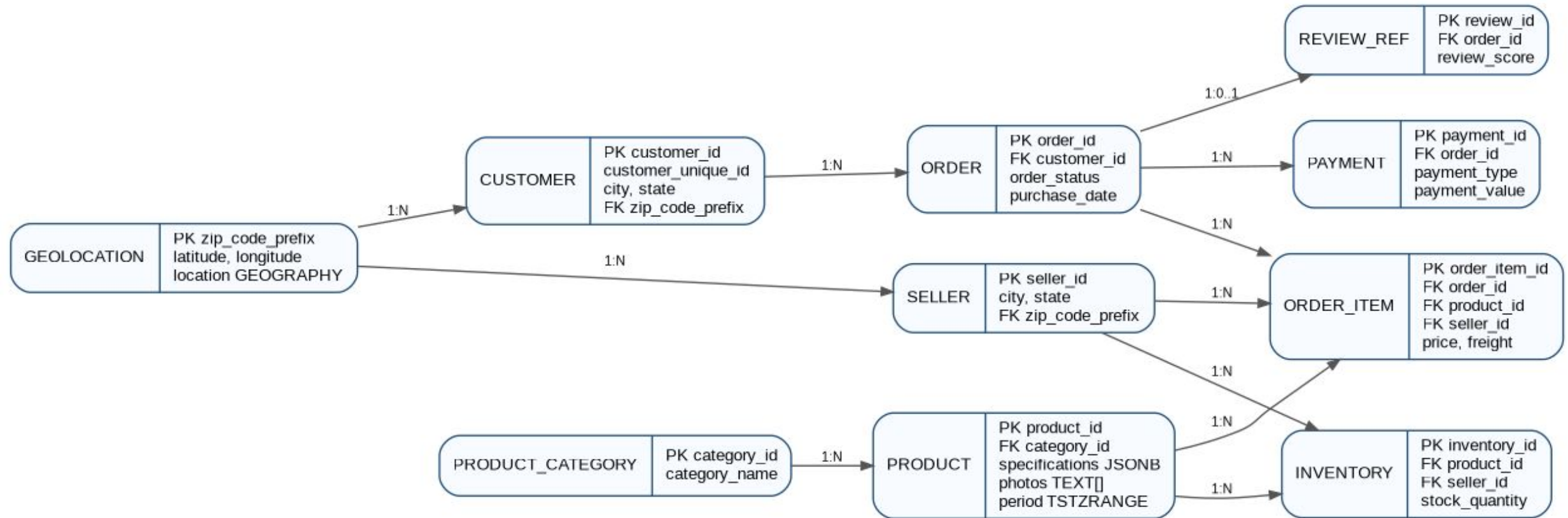
Búsqueda y geografía

Texto tolerante a errores y análisis de ubicación.

RNF	Métrica objetivo
Disponibilidad	99,5 % mensual para el prototipo académico en Supabase.
Rendimiento OLTP	Pedido por order_id <= 100 ms; pedidos por cliente <= 300 ms; insertar orden <= 500 ms.
Rendimiento OLAP	Dashboards agregados <= 2 s; particiones históricas <= 5 s.
Mantenibilidad	Diccionario, scripts SQL, EDA y monitoreo con EXPLAIN ANALYZE.

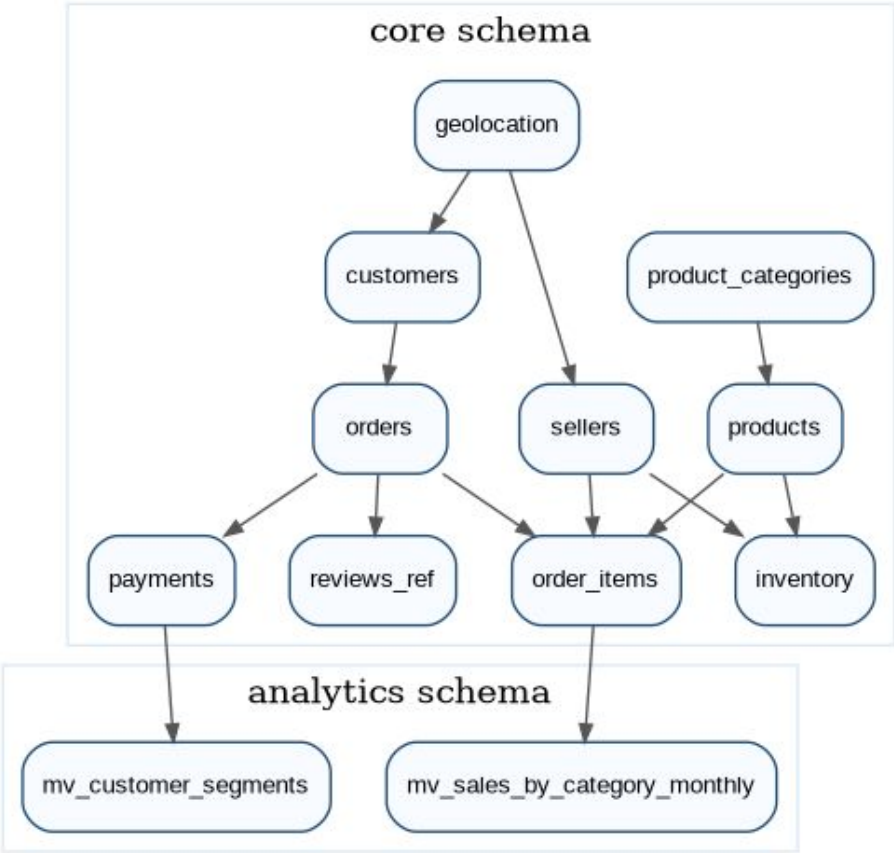
3. Diseño conceptual: modelo ER

Entidades, relaciones, cardinalidades, PK y FK para un modelo normalizado.



4. Esquema relacional PostgreSQL

Modelo normalizado mínimo en 3FN para el núcleo transaccional.



Grupo	Tablas / propósito
Maestras	customers, sellers, product_categories, products, geolocation
Transaccionales	orders, order_items, payments, inventory, reviews_ref
Analíticas	analytics.mv_sales_by_category_monthly, analytics.mv_customer_segments
Control	PK, FK, UNIQUE, CHECK, dominios, ENUM, timestamps de auditoría

5. PostgreSQL avanzado

Tipos nativos, extensiones y optimizaciones para cargas híbridas.

Elemento	Uso	Justificación
JSONB	products.specifications	Atributos variables por categoría.
TEXT[]	products.photos	Múltiples fotos por producto.
TSTZRANGE	products.promotion_period	Vigencia promocional consultable.
PostGIS	geolocation.location	Distancias vendedor-cliente.
pg_trgm	product_name	Búsqueda tolerante a errores.
pgcrypto	UUID	Identificadores seguros.

Particionamiento

orders particionada por purchase_date: hot/cold partitions para datos recientes e históricos.

Vistas materializadas

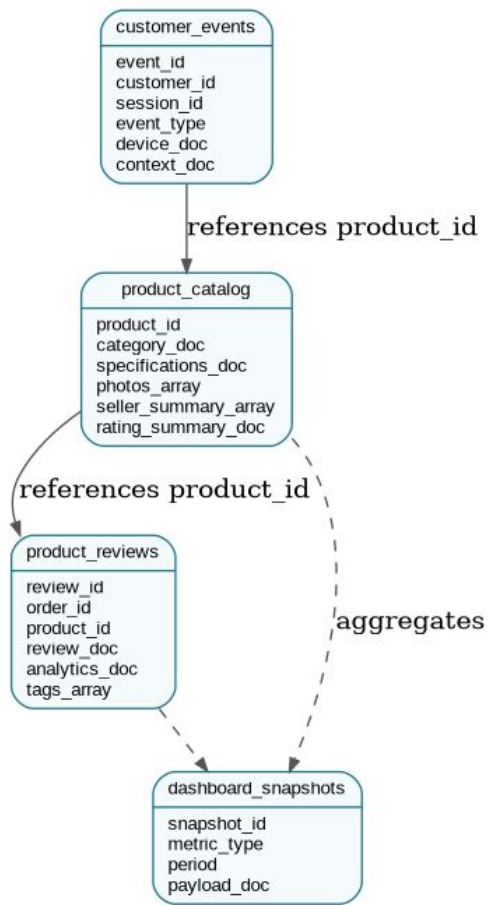
mv_sales_by_category_monthly y mv_customer_segments para tableros OLAP.

Triggers updated_at

Auditoría automática sobre tablas críticas.

6. Diseño lógico preliminar MongoDB

Repositorio complementario para documentos flexibles y analítica de lectura.



Colección	Patrón	Propósito
product_catalog	Embedding + atributos flexibles	Catálogo enriquecido y búsqueda.
product_reviews	Reference + documento extendido	Texto, sentimiento y keywords.
customer_events	Append-only events	Navegación, funnels y recomendaciones.
dashboard_snapshots	Precomputed views	Lecturas rápidas no críticas.

7. Matriz de decisión PostgreSQL vs MongoDB

Selección por criticidad, estructura, consistencia y patrón de acceso.

Entidad / módulo	PostgreSQL	MongoDB	Decisión
Pedidos, pagos, inventario	Muy alto	Bajo	PostgreSQL por ACID, FK y constraints.
Clientes y vendedores	Alto	Bajo	PostgreSQL como dato maestro.
Productos	Alto	Medio	Maestro relacional + catálogo documental.
Reseñas	Medio	Alto	Referencia relacional + documento extendido.
Eventos de usuario	Bajo	Muy alto	MongoDB por volumen y variabilidad.
Dashboards	Medio	Medio	MVs en PostgreSQL + snapshots opcionales.

8. Análisis CAP y trade-offs

La arquitectura prioriza consistencia donde hay impacto financiero y flexibilidad donde hay datos variables.

PostgreSQL

Prioriza consistencia: transacciones ACID, integridad referencial, validaciones y control de concurrencia.

MongoDB

Prioriza disponibilidad/flexibilidad: esquema adaptable, documentos enriquecidos y consistencia eventual.

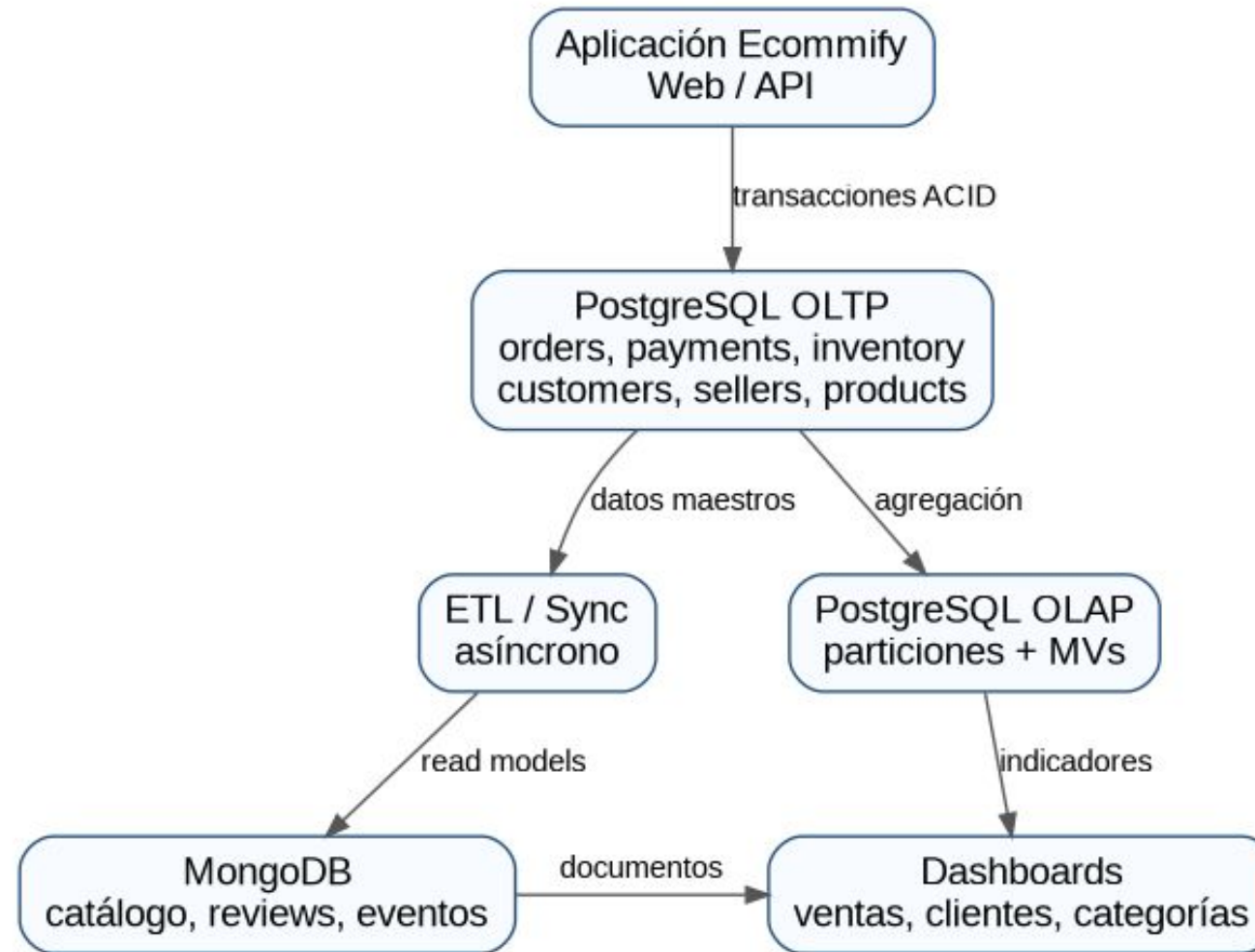
Sincronización

ETL/Sync asíncrono desde datos maestros hacia modelos de lectura documental.

Trade-off	Riesgo	Mitigación
JSONB en productos	Validación más compleja	Reglas por categoría y constraints parciales.
Particionamiento orders	Mayor operación	Automatizar particiones mensuales.
Vistas materializadas	Datos no siempre en tiempo real	Refresh programado y métricas OLAP.
MongoDB complementario	Consistencia eventual	IDs maestros y sincronización controlada.

9. Arquitectura híbrida transaccional-analítica

Flujo de datos operacional, analítico y documental.



10. Limitaciones, mitigaciones y plan de implementación

Implementación progresiva y validación técnica del prototipo.

Limitación	Mitigación
Supabase Free Tier	Índices selectivos, particiones y pruebas controladas.
MongoDB Atlas M0	Colecciones acotadas y documentos compactos.
Google Colab	EDA reproducible y datasets versionados.
Volumen histórico	Hot/cold partitions y vistas materializadas.

Fase	Resultado
1. EDA y calidad	Relaciones, nulos, volúmenes y patrones temporales/geográficos.
2. DDL PostgreSQL	Extensiones, schemas, tipos, tablas, constraints, índices.
3. OLTP/OLAP	Particiones, triggers, MVs y queries de validación.
4. MongoDB	Schemas JSON, documentos ejemplo e índices.
5. Validación	EXPLAIN ANALYZE, métricas, README y repositorio GitHub.